Quality per Joule: A Comparative Study of Energy Efficiency in Autoencoders, VAEs, GANs, WGAN-GPs, and Diffusion Models

---

Md. Masud Mazumder

ECE 7650 – Deep Generative Modeling: Theory & Application

Winter 2026

GitHub Repository: **Link**

Date: March 31, 2026

Acknowledgment of AI Assistance

ChatGPT were used to assist in organizing the structure of this document and reviewing relevant background concepts related to generative models and energy measurement. Code conversion from the provided notebook to PyTorch was initially done using ChatGPT and later inspected and fixed by the author. In some cases, text written by the author was polished further by ChatGPT for better presentation and enhanced readability. All ideas, experimental design decisions, implementation, and final written content reflect the author's own understanding and academic work.

Contents

1 Introduction **4**

1.1 Background and Motivation 4

1.2 Problem Statement 5

1.3 Proposed Metric: Quality per Joule 5

1.4 Objectives 5

1.5 Summary of Key Findings 6

1.6 Report Organization 6

2 Background and Related Work **8**

2.1 Autoencoders 8

2.2 Variational Autoencoders 8

2.3 Generative Adversarial Networks 9

2.4 Wasserstein GAN with Gradient Penalty 9

2.5 Denoising Diffusion Probabilistic Models 10

2.6 Fréchet Inception Distance 10

2.7 Energy Measurement in Deep Learning 11

2.8 Related Work on Green AI 11

3 Methodology **12**

3.1 Dataset 12

3.2 Experimental Design Principles 13

3.3 Evaluation Metrics 13

3.3.1 Quality Metrics 14

3.3.2 Efficiency Metrics 14

3.4 Quality per Joule Framework 14

4 Implementation Details **15**

4.1 Hardware and Software Environment 15

4.2 Model Architectures 15

4.2.1 Convolutional Autoencoder 16

4.2.2 Variational Autoencoder 16

4.2.3 DCGAN 16

4.2.4 WGAN-GP 16

4.2.5 DDPM 16

4.3 Training Configurations 17

4.4 Energy Measurement Setup 17

5 Experimental Results and Analysis **18**
5.1 Training Dynamics 18
5.2 Generated Samples 19
5.3 Generative Quality (FID) 20
5.4 Energy Consumption 21
5.5 Quality per Joule Comparison 22
5.6 FID vs. Energy Trade-off Analysis 24
5.7 Discussion 25

6 Conclusion and Future Work **27**
6.1 Summary of Contributions 27
6.2 Limitations 27
6.3 Future Work 28

Chapter 1

Introduction

1.1 Background and Motivation

Deep generative models have become one of the most active areas of research in modern machine learning, enabling a wide range of applications from image synthesis and data augmentation to drug discovery and creative content generation. Over the past decade, several foundational architectures have emerged, each offering distinct approaches to modeling complex data distributions. Autoencoders (AEs) learn compact latent representations through deterministic encoding and reconstruction. Variational Autoencoders (VAEs) extend this framework by imposing a probabilistic structure on the latent space, enabling principled sampling and generation. Generative Adversarial Networks (GANs) employ an adversarial training paradigm between a generator and a discriminator, often producing sharper images but suffering from training instability. Wasserstein GANs with Gradient Penalty (WGAN-GPs) address several of these stability issues by replacing the Jensen–Shannon divergence with the Wasserstein distance and enforcing a Lipschitz constraint through gradient penalty. Finally, Denoising Diffusion Probabilistic Models (DDPMs) generate samples through iterative denoising, achieving state-of-the-art image quality at the cost of significantly longer sampling and training times.

While extensive research has focused on improving the generative quality of these models, comparatively little attention has been devoted to understanding their energy consumption and computational efficiency. As generative AI systems are increasingly deployed at scale in both research and industry, their environmental and economic costs have become an important consideration. Training large generative models requires substantial GPU compute, translating directly into significant energy consumption and carbon emissions. With growing awareness of the environmental impact of artificial intelligence, there is a clear need for evaluation frameworks that go beyond output quality and also account for the computational resources consumed during training.

This motivation is further reinforced by practical constraints: in many real-world scenarios, practitioners operate under limited computational budgets, restricted hardware access, or sustainability mandates. In such settings, selecting a generative model purely on the basis of output quality is insufficient. A model that achieves marginally better image fidelity but consumes an order of magnitude more energy may not be the optimal choice. Thus, an energy-aware evaluation framework becomes essential for making informed model selection decisions.

1.2 Problem Statement

Generative models such as AEs, VAEs, GANs, WGAN-GPs, and DDPMs differ substantially in their architectural design, loss functions, optimization dynamics, and sampling procedures. These differences lead to significant variation in both the quality of generated outputs and the computational resources required to train them. However, the standard evaluation paradigm for generative models focuses almost exclusively on output quality metrics such as Fréchet Inception Distance (FID), while treating computational cost as a secondary concern or ignoring it entirely.

This project addresses the following question: **How do the five major families of deep generative models compare when evaluated jointly on generative quality and energy efficiency?** Specifically, this study measures the FID score and total GPU energy consumption for each model under controlled experimental conditions–identical dataset (CelebA at 32×32), comparable architectures, same batch size (64), same number of epochs (30), and same hardware–and proposes a unified metric, **Quality per Joule (QpJ)**, to quantify the generative efficiency of each model.

1.3 Proposed Metric: Quality per Joule

To enable a principled comparison of generative quality and energy cost, this study introduces the Quality per Joule metric, defined as:

$$\text{QpJ} = \frac{1}{\text{FID} \times \text{Energy}} \tag{1.1}$$

where FID is the Fréchet Inception Distance of the generated samples (lower is better) and Energy is the total GPU energy consumed during training, measured in Joules. This formulation rewards models that simultaneously achieve low FID (high quality) with minimal energy expenditure. A higher QpJ value indicates a more energy-efficient model relative to its generative quality. By combining both dimensions into a single scalar, QpJ allows direct comparison across model families that may occupy very different regions of the quality–energy space.

1.4 Objectives

The primary objectives of this study are as follows:

- Implement and train lightweight, comparable architectures for five generative model families–Autoencoder, VAE, DCGAN, WGAN-GP, and DDPM–on the CelebA dataset at 32×32 resolution.

- Measure GPU energy consumption during training using the CodeCarbon energy tracking framework and NVIDIA's NVML interface with per-epoch granularity.
- Evaluate the generative quality of each model using Fréchet Inception Distance (FID) computed on 10,000 generated samples compared against a fixed reference set.
- Compute the proposed Quality per Joule metric for each model and analyze the energy-quality trade-off across model families.
- Develop insights that can inform sustainable model selection in resource-constrained or environmentally conscious deployment scenarios.

1.5 Summary of Key Findings

The experiments conducted in this study reveal substantial differences in both generative quality and energy consumption across the five model families. The key findings are summarized in Table 1.1.

Table 1.1: Summary of experimental results across five generative model families trained on CelebA (32×32) for 30 epochs.

Model	FID	Energy (kJ)	Time (min)	Params	QpJ ($\times 10^{-7}$)
Autoencoder	33.45	103.9	12.2	1.13M	**2.900**
VAE	117.81	108.3	12.7	1.65M	0.800
DCGAN	15.20	1,115.1	77.6	3.59M	0.600
WGAN-GP	20.52	4,466.2	320.6	5.21M	0.100
DDPM	20.30	1,098.0	84.5	15.82M	0.400

As shown in Table 1.1, the Autoencoder achieves the highest Quality per Joule score (2.900×10^{-7}), combining a moderate FID of 33.45 with the lowest energy consumption at 103.9 kJ. The DCGAN achieves the best FID of 15.20 but at roughly $10\times$ the energy cost, yielding a QpJ of 0.600×10^{-7}. The VAE consumes similar energy to the Autoencoder (108.3 kJ) but produces a substantially higher FID of 117.81, reflecting the inherent blurriness of VAE-generated samples due to KL regularization. The WGAN-GP achieves competitive quality (FID 20.52) but consumes over 4,466 kJ—more than $42\times$ the Autoencoder—due to its multi-step critic training. The DDPM achieves near-identical FID to WGAN-GP (20.30) at only one quarter of the energy cost.

1.6 Report Organization

The remainder of this report is organized as follows. **Chapter 2** provides the theoretical background on each generative model family, the FID metric, energy measurement methodology,

and reviews related work on green and energy-efficient AI. **Chapter 3** describes the experimental methodology, including the dataset, experimental design principles, evaluation metrics, and the Quality per Joule framework. **Chapter 4** presents the implementation details, covering model architectures, training configurations, hyperparameters, hardware specifications, and energy measurement tooling. **Chapter 5** reports the experimental results and provides a detailed comparative analysis of generative quality, energy consumption, and the Quality per Joule metric across all five models. **Chapter 6** concludes the report with a summary of findings, discussion of limitations, and suggestions for future work.

Chapter 2

Background and Related Work

This chapter provides the theoretical foundations for the five generative model families studied in this project, describes the evaluation metrics used, and reviews related work on energy-efficient and sustainable deep learning.

2.1 Autoencoders

Autoencoders (AEs) [1] are neural networks trained to learn a compressed latent representation of input data by mapping inputs through an encoder to a low-dimensional bottleneck and then reconstructing the original input through a decoder. The training objective is to minimize the reconstruction error between the input $\mathbf{x}$ and the reconstructed output $\hat{\mathbf{x}}$, typically measured using Mean Squared Error (MSE):

$$\mathscr{L}_{\text{AE}} = \frac{1}{N} \sum \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \tag{2.1}$$

Because AEs use a deterministic mapping and do not impose any distributional constraints on the latent space, sampling from the latent space for generation is not straightforward. However, AEs are computationally efficient, easy to train, and serve as a useful baseline for comparing the energy costs and reconstruction capabilities of more sophisticated generative models. In this study, a convolutional autoencoder is used as the baseline model.

2.2 Variational Autoencoders

Variational Autoencoders (VAEs), introduced by Kingma and Welling [2], extend the autoencoder framework by imposing a probabilistic structure on the latent space. Instead of encoding each input to a single point, the encoder outputs the parameters (mean μ and variance σ^2) of a Gaussian distribution. The training objective combines reconstruction loss with a Kullback–Leibler (KL) divergence regularizer:

$$\mathscr{L}_{\text{VAE}} = \mathbb{E}\left[\|\mathbf{x} - \hat{\mathbf{x}}\|^2\right] + D_{\text{KL}}\left(q(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\right) \tag{2.2}$$

The KL divergence term encourages the learned posterior $q(\mathbf{z}|\mathbf{x})$ to remain close to a standard normal prior $p(\mathbf{z}) = \mathscr{N}(\mathbf{0}, \mathbf{I})$, ensuring that the latent space is smooth and continuous. This

enables meaningful sampling: new data points can be generated by drawing latent vectors from the prior and passing them through the decoder. The reparameterization trick $\mathbf{z} = \mu + \varepsilon \cdot \sigma$, $\varepsilon \sim \mathcal{N}(0,1)$ enables end-to-end gradient flow through the stochastic sampling step. However, the KL regularization often leads to blurrier outputs compared to adversarially trained models, as the reconstruction objective tends to average over modes of the data distribution.

2.3 Generative Adversarial Networks

Generative Adversarial Networks (GANs), proposed by Goodfellow et al. [3], consist of two networks–a generator G and a discriminator D–trained in a minimax game. The generator maps random noise $\mathbf{z} \sim p(\mathbf{z})$ to generated samples, while the discriminator attempts to distinguish real data from generated data. The original GAN objective is:

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{2.3}$$

In practice, the Deep Convolutional GAN (DCGAN) architecture, introduced by Radford et al. [4], established a set of architectural guidelines for stable GAN training using convolutional layers: replacing pooling with strided convolutions, using batch normalization in both the generator and discriminator, removing fully connected hidden layers, using ReLU activation in the generator and LeakyReLU in the discriminator, and using label smoothing for the real labels. This project uses a DCGAN implementation as the representative GAN architecture.

2.4 Wasserstein GAN with Gradient Penalty

Standard GANs can suffer from training instability, mode collapse, and vanishing gradients when the discriminator becomes too strong. The Wasserstein GAN (WGAN) addresses these issues by replacing the Jensen–Shannon divergence with the Earth Mover's (Wasserstein-1) distance as the training objective. The WGAN-GP variant, proposed by Gulrajani et al. [5], enforces the required Lipschitz constraint through a gradient penalty term rather than weight clipping:

$$\mathscr{L}_C = \mathbb{E}[C(G(\mathbf{z}))] - \mathbb{E}[C(\mathbf{x})] + \lambda \cdot \mathbb{E}\left[(\|\nabla_{\hat{\mathbf{x}}} C(\hat{\mathbf{x}})\|_2 - 1)^2\right] \tag{2.4}$$

Here, C denotes the critic (replacing the discriminator), $\hat{\mathbf{x}}$ represents points sampled uniformly along straight lines between real and generated data pairs, and λ (set to 10 in this study) controls the gradient penalty strength. The critic is trained for multiple iterations ($n_{\text{critic}} = 3$ in this study) per generator update. Batch normalization is removed from the critic because it creates inter-sample dependencies that corrupt the per-sample gradient computations required by the penalty term. While WGAN-GP produces more stable training and often better generative

quality, the multiple critic updates per generator step significantly increase the computational cost per epoch.

2.5 Denoising Diffusion Probabilistic Models

Denoising Diffusion Probabilistic Models (DDPMs), introduced by Ho et al. [6], define a forward diffusion process that gradually adds Gaussian noise to data over T timesteps, and learn a reverse denoising process to generate samples from pure noise. The forward process at timestep t is defined as:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t;\, \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\, \beta_t \mathbf{I}\right) \tag{2.5}$$

The model learns to predict the noise ε added at each timestep, and the training objective is a simplified mean squared error between the predicted and actual noise:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,\mathbf{x}_0,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(\mathbf{x}_t, t)\|^2\right] \tag{2.6}$$

During sampling, the model iteratively denoises starting from pure Gaussian noise, requiring T sequential forward passes through the network. While DDPMs have achieved state-of-the-art generative quality in many benchmarks, their iterative nature makes both training and sampling computationally expensive. In this study, $T = 1{,}000$ timesteps are used during training and FID evaluation with a linear noise schedule from $\beta_{\text{start}} = 10^{-4}$ to $\beta_{\text{end}} = 0.02$.

2.6 Fréchet Inception Distance

Fréchet Inception Distance (FID), introduced by Heusel et al. [7], is the most widely used quantitative metric for evaluating the quality of generated images. FID computes the distance between the feature distributions of real and generated images, where features are extracted from a pretrained Inception-v3 network. Formally:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\right) \tag{2.7}$$

where (μ_r, Σ_r) and (μ_g, Σ_g) are the mean and covariance of the Inception features for the real and generated distributions, respectively. Lower FID scores indicate that the distribution of generated images is closer to the distribution of real images, implying higher generative quality. FID captures both the fidelity (individual image quality) and diversity (coverage of the data distribution) of generated samples. In this study, FID is computed using 10,000 generated samples compared against a fixed subset of 10,000 images from the CelebA validation split.

2.7 Energy Measurement in Deep Learning

GPU energy consumption during model training can be measured using hardware-level interfaces. NVIDIA's Management Library (NVML) provides real-time access to GPU power draw in watts through tools such as `nvidia-smi`. The total energy consumed during training is computed as:

$$\text{Energy (J)} = \text{Average Power (W)} \times \text{Training Time (s)} \tag{2.8}$$

The CodeCarbon library [12] provides a convenient Python wrapper around NVML, automatically tracking GPU power draw over time and computing total energy consumption and estimated CO_2 emissions. Idle power subtraction is used to isolate the energy attributable to the training workload itself, excluding the baseline power consumption of the GPU at rest. This study uses CodeCarbon for all energy measurements, with additional per-epoch energy logging through custom NVML-based instrumentation.

2.8 Related Work on Green AI

The concept of Green AI was formalized by Schwartz et al. [9], who argued that the AI research community should prioritize computational efficiency alongside accuracy, and proposed reporting the computational cost of experiments as a standard practice. Their work distinguished between *Red AI*, which pursues state-of-the-art results regardless of computational cost, and *Green AI*, which seeks to achieve good results with reduced computational resources.

Strubell et al. [10] estimated the carbon footprint of training large NLP models and found that the energy required for some training runs was equivalent to the lifetime carbon emissions of multiple automobiles. Patterson et al. [11] provided a comprehensive analysis of the energy consumption of machine learning training workloads at Google, highlighting the importance of hardware selection, data center efficiency, and algorithmic choices. More recently, several studies have begun to examine the energy costs of generative models specifically, though systematic comparisons across multiple generative families under controlled conditions remain rare. This project aims to contribute to this emerging body of work by providing a controlled, fair comparison with a unified metric.

Chapter 3

Methodology

This chapter describes the experimental methodology used in this study, including the dataset, experimental design principles, evaluation metrics, and the Quality per Joule framework.

3.1 Dataset

All experiments use the CelebA (CelebFaces Attributes) dataset [8], which contains over 200,000 celebrity face images with 40 binary attribute annotations. Only the images are used in an unconditional generative setting (no attribute labels). The standard train/validation/test splits are employed: approximately 162,770 training images, 19,867 validation images, and 19,962 test images.

Preprocessing consists of: (1) center-cropping to 178×178 pixels to remove background padding and retain the face region (standard for CelebA, as raw images are 218×178 with faces centered), (2) resizing to 32×32, (3) optional random horizontal flipping during training, and (4) normalization to the $[-1, 1]$ range. All models receive identically preprocessed images. CelebA was chosen because it is a well-established benchmark for face generation that is large enough to reveal meaningful differences in generative quality while remaining feasible for multiple complete training runs within a reasonable energy budget.

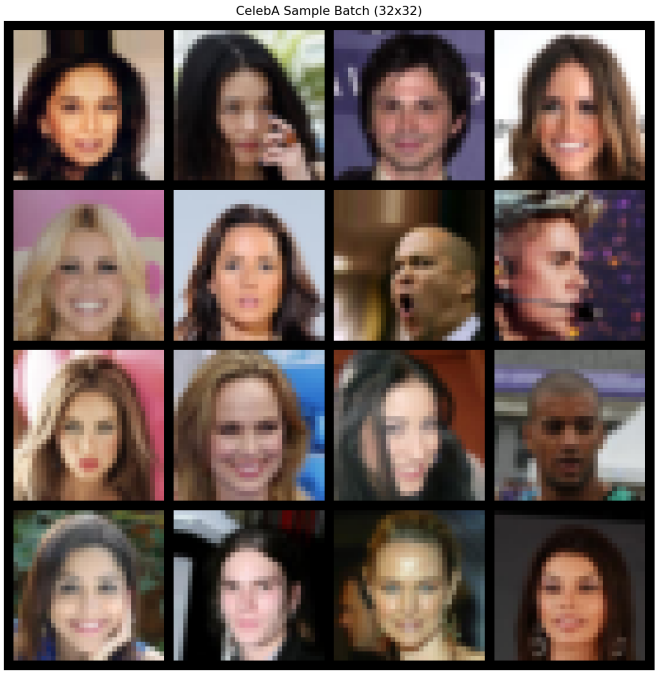

Figure 3.1: Sample batch from the CelebA dataset after preprocessing (32×32, normalized to $[-1, 1]$).

3.2 Experimental Design Principles

To ensure a fair comparison, the following experimental controls are enforced across all five models:

- **Identical dataset and preprocessing:** All models are trained on the same CelebA training split with the same preprocessing pipeline.
- **Identical batch size:** A batch size of 64 is used for all models.
- **Same number of epochs:** All models are trained for exactly 30 epochs.
- **Same hardware:** All experiments are conducted on the same GPU to ensure consistent energy measurement.
- **Fixed random seed:** A seed of 42 is used for all experiments to ensure reproducibility.
- **Same evaluation protocol:** FID is computed using 10,000 generated samples compared against a fixed reference set of 10,000 validation images for all models.
- **Sequential execution with idle gaps:** A 10-second idle period is inserted between training stages to prevent residual GPU activity from bleeding into subsequent energy measurements.

3.3 Evaluation Metrics

3.3.1 Quality Metrics

The primary quality metric is the Fréchet Inception Distance (FID), computed at the 2048-dimensional Inception-v3 feature level on 10,000 generated samples compared against the CelebA validation reference set. For the Autoencoder, reconstruction MSE on the test set is additionally reported as a model-specific quality measure.

3.3.2 Efficiency Metrics

The following efficiency metrics are collected for each model:

- **Total training time** in seconds, measured wall-clock.
- **Average GPU power draw** in watts, measured via NVML through CodeCarbon.
- **Total GPU energy consumption** in Joules, computed as average power $\times$ total time.
- **Per-epoch energy consumption** in Joules, logged via custom NVML instrumentation.
- **Estimated CO_2 emissions** in grams, reported by CodeCarbon based on the regional carbon intensity of the Manitoba, Canada electricity grid.

3.4 Quality per Joule Framework

The Quality per Joule (QpJ) metric combines generative quality and energy cost into a single scalar:

$$\text{QpJ} = \frac{1}{\text{FID} \times \text{Energy}} \tag{3.1}$$

This metric is designed so that a higher value indicates a better energy–quality trade-off. Models with low FID (high quality) and low energy consumption will score highest. The metric penalizes both poor quality (high FID) and high energy cost multiplicatively, capturing the intuition that neither dimension should be optimized in isolation. In practice, QpJ values are very small (on the order of 10^{-7} to 10^{-8}) due to the large magnitudes of both FID scores and energy values in Joules. Results are therefore reported in units of $\times 10^{-7}$ for readability.

Chapter 4

Implementation Details

This chapter presents the specific implementation choices, model architectures, training configurations, and energy measurement setup used in the experiments.

4.1 Hardware and Software Environment

All experiments were conducted on a single workstation to ensure consistent and comparable energy measurements. Table 4.1 lists the hardware and software specifications.

Table 4.1: Hardware and software environment.

Component	Specification
CPU	Intel Core i7-9700K @ 3.60 GHz (8 cores)
GPU	NVIDIA GeForce RTX 2080 SUPER (8 GB)
RAM	31.06 GB
OS	Linux (Ubuntu, kernel 6.8.0-106)
Python	3.10.12
Framework	PyTorch
Energy Tracker	CodeCarbon 3.2.5
Location	Manitoba, Canada

The software environment includes Python 3.10 with PyTorch as the deep learning framework, torchvision for dataset loading and the Inception-v3 backbone, torchmetrics for FID computation, and the CodeCarbon library for energy tracking. GPU power draw is accessed through NVIDIA's NVML interface, which CodeCarbon wraps transparently. The average GPU power draw observed across all experiments ranged from approximately 113 W (AE, VAE) to 227 W (DCGAN).

4.2 Model Architectures

All five models use convolutional architectures designed for 32×32 RGB images. They are mainly taken from the implementation provided in the class. However, all of them are converted into PyTorch, and some modifications are incorporated to feed the dataset of fixed shape. The following subsections describe each architecture.

4.2.1 Convolutional Autoencoder

The encoder consists of five convolutional blocks (Conv2d $\rightarrow$ BatchNorm $\rightarrow$ LeakyReLU) that progressively downsample the 32×32 input through channel dimensions of $64 \rightarrow 128 \rightarrow 256 \rightarrow 256 \rightarrow 256$ to a 4×4 feature map, followed by a fully connected layer projecting to a 128-dimensional latent vector. The decoder mirrors this structure using transposed convolutions with a final Tanh activation mapping outputs to $[-1, 1]$. The model is trained with MSE reconstruction loss. Total parameters: **1,128,451**.

4.2.2 Variational Autoencoder

The VAE shares the same convolutional backbone as the Autoencoder but replaces the single latent projection with two parallel fully connected heads producing μ and $\log \sigma^2$ for a 128-dimensional latent space. The reparameterization trick $\mathbf{z} = \mu + \varepsilon \cdot \sigma$ is used to sample latent vectors during training. The loss function combines MSE reconstruction loss with KL divergence. Total parameters: **1,652,867**.

4.2.3 DCGAN

The DCGAN follows the architectural guidelines of Radford et al. [4]. The generator maps a 128-dimensional noise vector through six transposed convolutional blocks (ConvTranspose2d $\rightarrow$ BatchNorm $\rightarrow$ ReLU) with channel dimensions $512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 3$ to produce 32×32 images, with a Tanh output. The discriminator uses six convolutional blocks with LeakyReLU and a Sigmoid output. One-sided label smoothing (0.9 for real labels) is applied. Total parameters: **3,585,732**.

4.2.4 WGAN-GP

The WGAN-GP uses the same generator architecture as the DCGAN but replaces the discriminator with a critic that outputs an unbounded scalar (no Sigmoid). Batch normalization is removed from the critic to avoid interfering with the gradient penalty computation, as BN creates inter-sample dependencies that corrupt the per-sample gradient required by the penalty. The gradient penalty coefficient λ is set to 10, and the critic is updated 3 times per generator update ($n_{\text{critic}} = 3$). This 3:1 update ratio is the primary driver of the substantially higher training time compared to the standard DCGAN. Total parameters: **5,211,847**.

4.2.5 DDPM

The DDPM uses a 5-level U-Net architecture with sinusoidal timestep embeddings, residual blocks with GroupNorm(8), and self-attention at the bottleneck resolution. The base channel count is 32 with a time embedding dimension of 128. The channel progression through the U-Net is $32 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 128$ with skip connections. The diffusion process uses $T =$

1,000 timesteps during training with a linear noise schedule from $\beta_{\text{start}} = 10^{-4}$ to $\beta_{\text{end}} = 0.02$. Total parameters: **15,820,355**.

4.3 Training Configurations

Table 4.2 summarizes the training hyperparameters for each model. All models use the Adam optimizer with model-specific β values.

Table 4.2: Training hyperparameters for each model.

Hyperparameter	AE	VAE	DCGAN	WGAN-GP	DDPM
Latent dim	128	128	128	128	—
Learning rate	2e-4	2e-4	2e-4	2e-4	2e-4
Batch size	64	64	64	64	64
Epochs	30	30	30	30	30
Optimizer	Adam	Adam	Adam	Adam	Adam
β_1/β_2	—	—	0.5/0.999	0.5/0.9	0.0/0.999
n_{critic}	—	—	—	3	—
λ (GP)	—	—	—	10	—
Label smoothing	—	—	0.9	—	—
Timesteps (T)	—	—	—	—	1,000

4.4 Energy Measurement Setup

Energy consumption is tracked using the CodeCarbon library [12], which polls GPU power draw through NVIDIA's NVML interface at regular intervals throughout training. For each model, CodeCarbon records the total energy consumed in Joules, the average power draw in Watts, the total training time in seconds, and an estimate of CO_2 emissions based on the regional carbon intensity of the Manitoba, Canada electricity grid. Per-epoch energy breakdowns are also logged by recording the epoch start and end times along with the average power draw during each epoch, enabling analysis of energy accumulation dynamics over the course of training. Idle power subtraction is applied to isolate the energy attributable to training computation. A 10-second idle gap is inserted between model training stages to ensure clean energy separation.

Chapter 5

Experimental Results and Analysis

This chapter presents the experimental results obtained from training all five generative models on CelebA at 32×32 resolution under the controlled conditions described in the preceding chapters. The analysis covers training dynamics, generative quality, energy consumption, and the Quality per Joule comparison.

5.1 Training Dynamics

Figure 5.1 shows the training loss curves for all five models across 30 epochs. Each model family exhibits characteristic training behavior consistent with its underlying optimization objective.

The Autoencoder shows a smooth, monotonically decreasing validation loss (MSE), converging from 0.0278 at epoch 1 to 0.0118 at epoch 30, indicating stable reconstruction learning throughout training. The VAE exhibits a similar convergence pattern in its combined loss (MSE + KL divergence), with the validation loss decreasing from 0.2046 to 0.1305. The reconstruction component decreases steadily while the KL divergence term stabilizes around 0.044, indicating that the model successfully learns to balance reconstruction fidelity with latent space regularization.

The DCGAN shows stable adversarial training dynamics: the generator loss (BCE) stabilizes around 0.83, while the discriminator loss converges near 1.36–close to the theoretical equilibrium of $\log 4 \approx 1.386$ for a perfectly calibrated discriminator. The WGAN-GP exhibits the characteristic Wasserstein critic/generator loss behavior, with the critic loss converging from -2.14 to -0.25 and the generator loss from -4.47 to 0.39, with the gradient penalty stabilizing near 0.04, demonstrating stable Wasserstein distance minimization without mode collapse. The DDPM noise prediction loss drops sharply during the first 5 epochs (from 0.178 to 0.032) and then gradually improves to 0.026, consistent with the model first learning coarse structure and then refining fine details.

Figure 5.1: Training loss curves for all five generative models over 30 epochs.

5.2 Generated Samples

Figures 5.2–5.6 show representative outputs generated by each model after 30 epochs of training. The Autoencoder output shows reconstructions of test images, while all other models show unconditionally generated samples from their respective learned distributions.

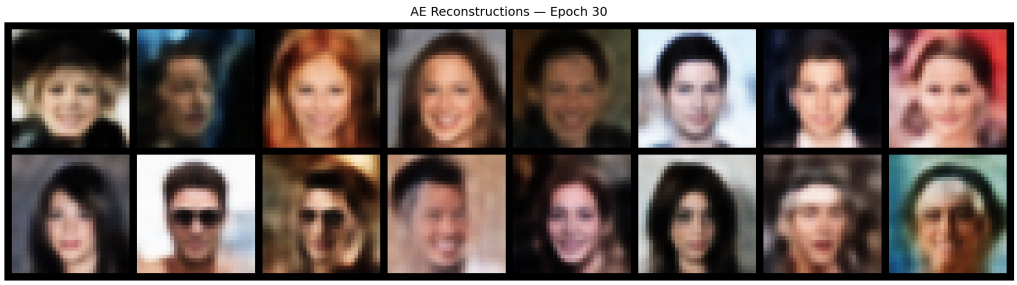

Figure 5.2: Autoencoder – Reconstructions (Epoch 30). The model produces faithful reconstructions with slight blurriness in fine details.

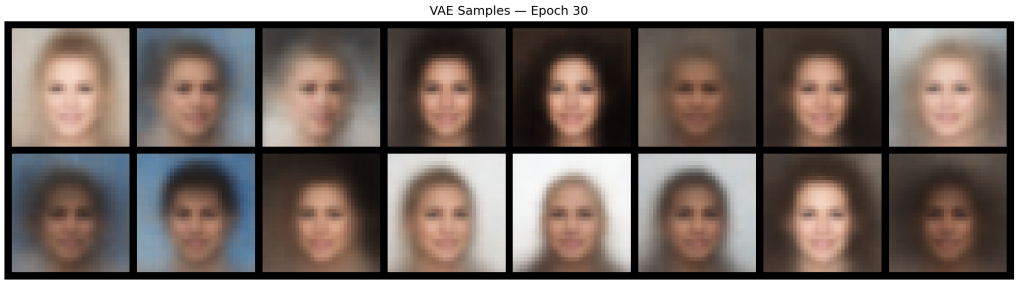

Figure 5.3: VAE – Generated Samples (Epoch 30). Samples are noticeably blurrier than GAN-based outputs, reflecting the averaging effect of KL regularization on the latent space.

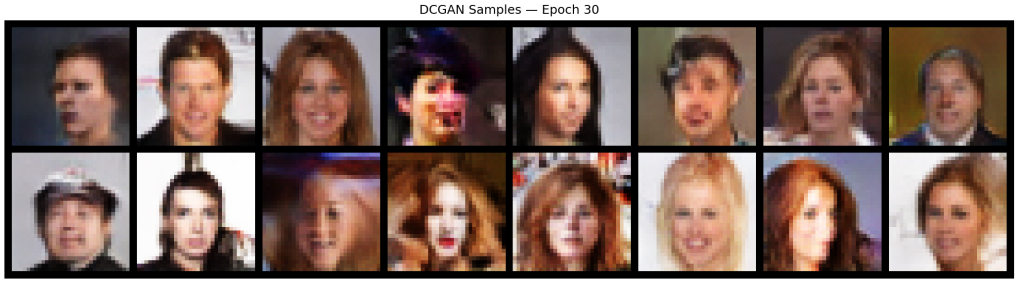

Figure 5.4: DCGAN – Generated Samples (Epoch 30). The model produces the sharpest and most realistic face images among all five models.

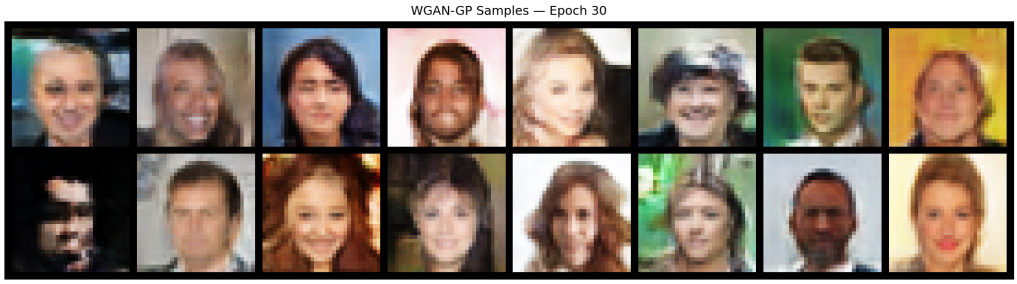

Figure 5.5: WGAN-GP – Generated Samples (Epoch 30). Samples are visually competitive with DCGAN, showing good facial structure and diversity.

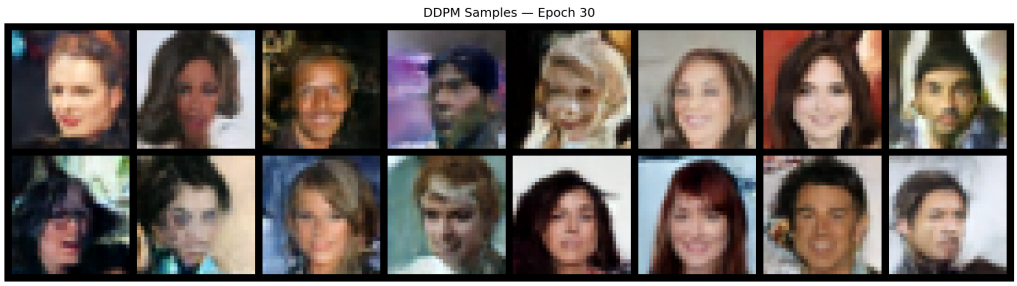

Figure 5.6: DDPM – Generated Samples (Epoch 30). The model produces high-quality samples with good diversity, demonstrating effective denoising even with reduced evaluation timesteps.

Visually, DCGAN and WGAN-GP produce the sharpest and most realistic face images. DDPM samples are also of high quality, showing good facial structure and diversity. The Autoencoder produces faithful reconstructions but exhibits slight blurriness in fine details such as hair texture and background elements. VAE samples are noticeably blurrier, reflecting the well-known tendency of VAEs to produce overly smooth outputs due to the KL regularization on the latent space.

5.3 Generative Quality (FID)

Table 5.1 presents the FID scores computed on 10,000 generated samples for each model.

Table 5.1: FID scores for all five models (lower is better).

Model	FID Score	Ranking
DCGAN	15.20	1 (Best)
DDPM	20.30	2
WGAN-GP	20.52	3
Autoencoder	33.45	4
VAE	117.81	5 (Worst)

The DCGAN achieves the best (lowest) FID of 15.20, followed closely by the DDPM at 20.30 and the WGAN-GP at 20.52. The Autoencoder produces a moderate FID of 33.45, reflecting its reconstruction-based approach, which does not optimize for distributional similarity. The VAE produces the highest FID of 117.81, consistent with the well-known tendency of VAEs to generate blurrier outputs due to the averaging effect of the reconstruction loss combined with KL regularization.

Notably, the DDPM and WGAN-GP achieve nearly identical FID scores despite fundamentally different generative mechanisms–iterative denoising versus adversarial training. This suggests that both approaches converge to similar quality levels for this dataset and training budget, though with very different energy implications (analyzed in subsequent sections).

5.4 Energy Consumption

Table 5.2 presents the detailed energy consumption metrics for each model. Figure 5.7 shows the cumulative training energy over epochs.

Table 5.2: Energy consumption metrics for all five models.

Model	Time (s)	Time (min)	Avg Power (W)	Energy (kJ)	CO_2 (g)
AE	732.73	12.2	112.86	103.93	0.152
VAE	762.47	12.7	113.02	108.32	0.159
DCGAN	4,653.64	77.6	226.68	1,115.12	1.634
WGAN-GP	19,236.63	320.6	214.15	4,466.16	6.543
DDPM	5,070.97	84.5	218.45	1,098.03	1.609

The AE and VAE are the most energy-efficient models, consuming only 103.93 kJ and 108.32 kJ respectively in approximately 12–13 minutes. These two reconstruction-based models operate at roughly half the GPU power draw ($\sim$113 W) compared to the GAN and diffusion models ($\sim$214–227 W), and their total energy consumption is an order of magnitude lower than the adversarial and diffusion models.

The DCGAN consumes 1,115.12 kJ in 77.6 minutes, approximately 10× more than the reconstruction-based models, driven by higher GPU utilization during adversarial training. The DDPM consumes a comparable 1,098.03 kJ in 84.5 minutes, reflecting a similar computational profile despite its fundamentally different training mechanism.

The WGAN-GP is by far the most energy-intensive model, consuming 4,466.16 kJ and requiring approximately 5.3 hours of training. This is directly attributable to the 3:1 critic-to-generator update ratio: each epoch of WGAN-GP training involves three times as many backward passes through the critic as a standard GAN epoch, plus the additional computational overhead of computing the gradient penalty at every critic update. The WGAN-GP consumes over 42× more energy than the Autoencoder for a FID improvement of only 13 points.

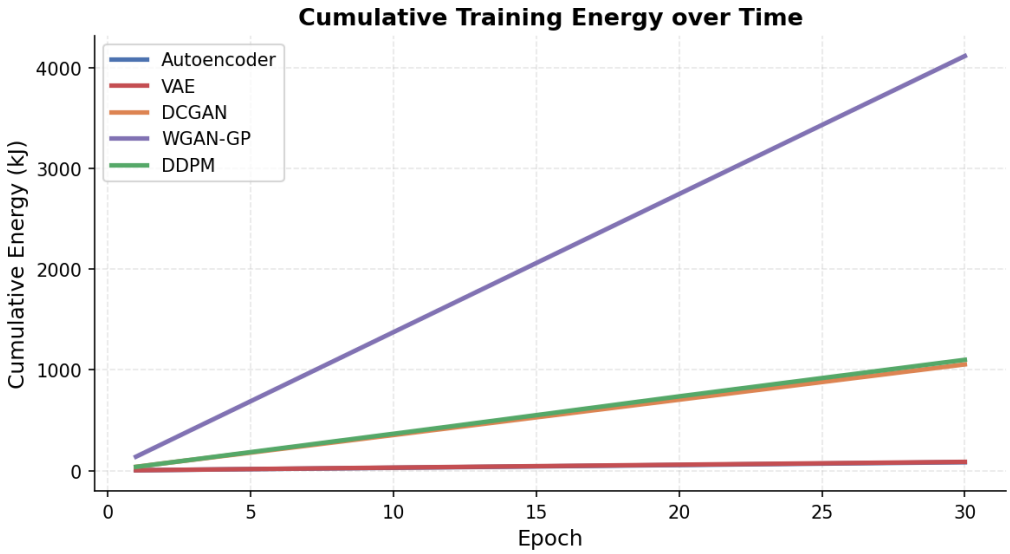

Figure 5.7: Cumulative training energy consumption over 30 epochs for all five models.

Figure 5.7 reveals that the energy growth is approximately linear for all models, as each epoch involves the same number of training iterations over the fixed dataset. The WGAN-GP's slope is dramatically steeper than all other models (∼137 kJ/epoch) due to its multiple critic updates per iteration. The DCGAN and DDPM accumulate energy at similar rates (∼35–36 kJ/epoch). The Autoencoder and VAE curves are nearly indistinguishable at the bottom of the chart (∼2.8–2.9 kJ/epoch), reflecting their similar and minimal computational profiles.

5.5 Quality per Joule Comparison

Table 5.3 presents the Quality per Joule metric for each model, and Figure 5.8 visualizes this comparison.

Table 5.3: Quality per Joule comparison across all five models (higher is better).

Model	FID	Energy (kJ)	QpJ ($\times 10^{-7}$)	Ranking
Autoencoder	33.45	103.9	2.900	1 (Best)
VAE	117.81	108.3	0.800	2
DCGAN	15.20	1,115.1	0.600	3
DDPM	20.30	1,098.0	0.400	4
WGAN-GP	20.52	4,466.2	0.100	5 (Worst)

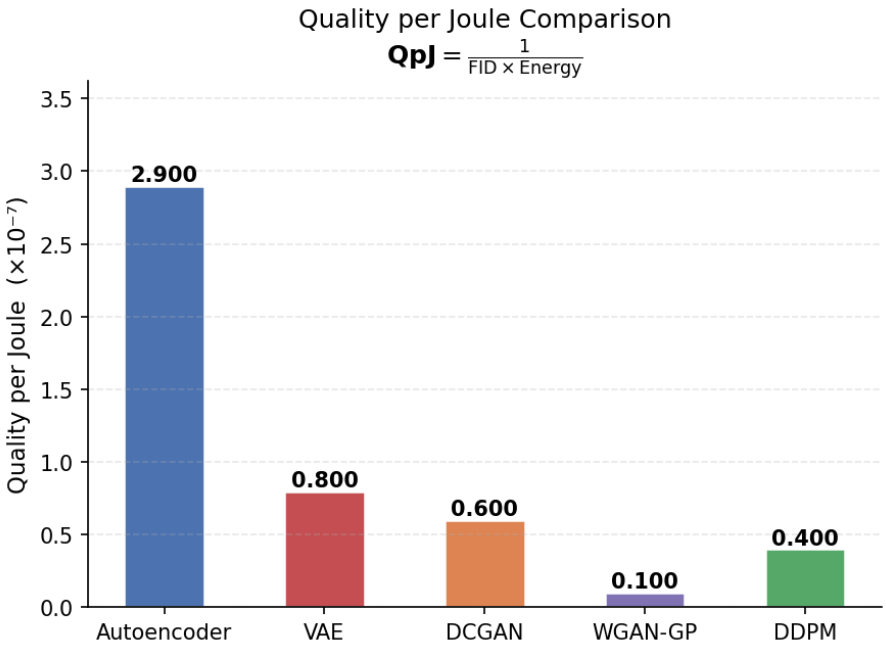

Figure 5.8: Quality per Joule comparison. The Autoencoder achieves the highest energy-quality efficiency, followed by VAE and DCGAN.

The Autoencoder achieves the highest QpJ of 2.900×10^{-7}–approximately 3.6$\times$ higher than the VAE and 4.8$\times$ higher than DCGAN–making it the most energy-efficient model relative to its generative quality. Despite not achieving the best FID score, its extremely low energy consumption (103.9 kJ) combined with a moderate FID of 33.45 gives it the best overall efficiency.

The VAE ranks second with a QpJ of 0.800×10^{-7}. Although it consumes nearly the same energy as the Autoencoder, its relatively poor FID of 117.81 reduces its efficiency score. The DCGAN ranks third with a QpJ of 0.600×10^{-7}, achieving the best FID of 15.20 but at approximately 10$\times$ the energy cost of the reconstruction-based models. The DDPM ranks fourth with a QpJ of 0.400×10^{-7}, achieving competitive quality at similar energy to the DCGAN. The WGAN-GP achieves the lowest QpJ of 0.100×10^{-7} due to its extreme energy consumption (4,466.2 kJ) combined with only marginally better FID than the DDPM.

5.6 FID vs. Energy Trade-off Analysis

Figure 5.9 presents a scatter plot of FID score versus total training energy, with the ideal region (low energy, low FID) in the upper-left corner. This visualization reveals the fundamental trade-off structure across model families.

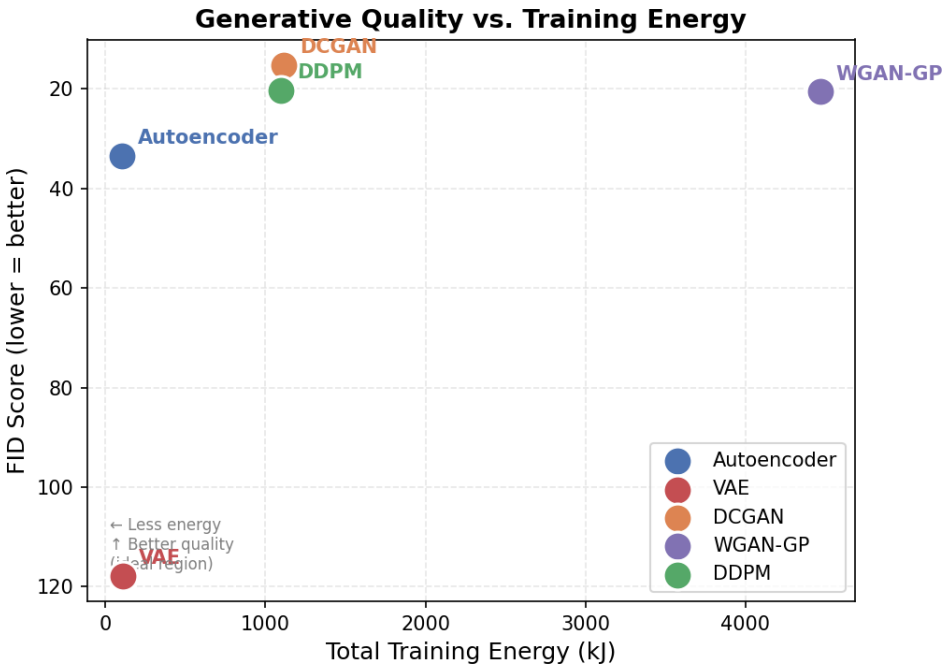

Figure 5.9: FID score vs. total training energy. The ideal operating region (low energy, low FID) is the upper-left corner. The y-axis is inverted so that better quality appears higher.

The scatter plot reveals three distinct clusters: (1) *Low energy, moderate quality*: AE and VAE, which consume $\sim$100 kJ but achieve higher FID scores; (2) *Moderate energy, high quality*: DCGAN and DDPM, which consume $\sim$1,100 kJ and achieve competitive FID scores around 15–20; and (3) *High energy, high quality*: WGAN-GP, which consumes 4$\times$ more energy than DCGAN for only marginally different FID. The VAE is an outlier in the low-energy cluster, as its high FID (117.81) reflects the inherent blurriness of VAE-generated samples rather than insufficient training.

No single model dominates all others in both dimensions simultaneously. The choice between models depends on the practitioner's priorities: if energy cost is the primary constraint, the Autoencoder is the optimal choice. If generative quality is paramount and energy is not a concern, the DCGAN delivers the best results. The DDPM offers a compelling middle ground–near-DCGAN quality at comparable energy–with superior training stability and no mode collapse risk.

5.7 Discussion

The full evaluation dashboard (Figure 5.10) provides a consolidated view of all metrics and trade-offs.

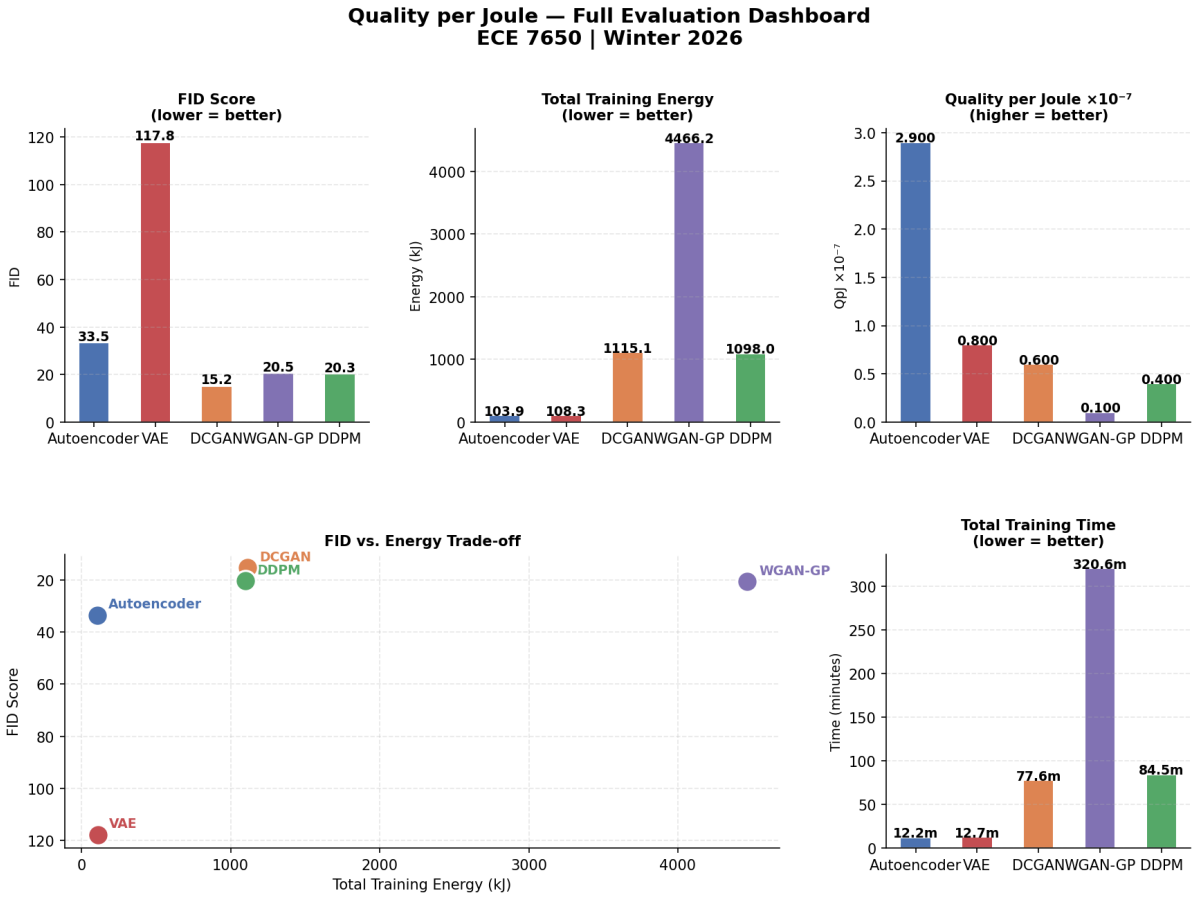

Figure 5.10: Full evaluation dashboard showing FID, energy, QpJ, FID vs. energy trade-off, and total training time across all five models.

Several key observations emerge from the experimental results. First, the relationship between generative quality and energy consumption is highly non-linear. The WGAN-GP achieves only a modest FID improvement over the DDPM (20.52 vs. 20.30) but consumes over 4× more energy (4,466 kJ vs. 1,098 kJ). This suggests severe diminishing returns: each unit of quality improvement becomes progressively more expensive in terms of energy.

Second, the DCGAN's strong performance under these experimental conditions is noteworthy. Despite having fewer parameters than the WGAN-GP (3.59M vs. 5.21M), the DCGAN achieves substantially better FID (15.20 vs. 20.52) at one-quarter the energy cost. This suggests that for this dataset and scale, the additional complexity of the Wasserstein training procedure does not yield quality gains commensurate with its energy overhead.

Third, the CO_2 emissions data, while small in absolute terms for these lightweight experiments,

reveals a $43\times$ difference between the most and least efficient models (0.152 g for AE vs. 6.543 g for WGAN-GP). When extrapolated to larger-scale models, larger datasets, and longer training schedules typical of production generative AI systems, these differences become environmentally significant.

Finally, the Quality per Joule metric successfully captures the efficiency trade-off in a single scalar. The ranking–Autoencoder $>$ VAE $>$ DCGAN $>$ DDPM $\gg$ WGAN-GP–provides a clear and actionable summary for practitioners making model selection decisions under energy constraints.

Chapter 6

Conclusion and Future Work

6.1 Summary of Contributions

This project presents a systematic comparative study of energy efficiency across five major families of deep generative models: Autoencoders, Variational Autoencoders, DCGANs, WGAN-GPs, and Denoising Diffusion Probabilistic Models. The study makes the following contributions:

- A controlled experimental comparison of five generative model families under identical conditions–same dataset (CelebA, 32×32), same batch size (64), same number of epochs (30), and same hardware (RTX 2080 SUPER).
- Comprehensive energy profiling using the CodeCarbon framework with per-epoch NVML-based power logging, providing detailed energy breakdowns and total energy consumption measurements for each model.
- A proposed unified metric, Quality per Joule (QpJ), that combines generative quality (FID) and energy efficiency into a single scalar for direct model comparison.
- An empirical demonstration that the energy–quality trade-off is highly non-linear: the WGAN-GP consumes over $42\times$ more energy than the Autoencoder for a FID improvement of only 13 points (33.45 to 20.52).
- Evidence that the Autoencoder provides the best energy-quality trade-off (highest QpJ = 2.9×10^{-7}), while the DCGAN provides the best absolute quality (FID = 15.20) at a moderate energy cost.
- A finding that DDPM achieves competitive FID (20.30) at comparable energy to DCGAN, offering a compelling quality–stability–energy trade-off.

6.2 Limitations

This study has several limitations that should be acknowledged:

- **Varying parameter counts:** Parameter counts vary across models (1.13M to 15.82M) due to the architectural requirements of each model family at 32×32 resolution. This prevents direct apples-to-apples capacity comparisons.

- **Single dataset:** Results are reported only on CelebA. Generalization to other datasets (e.g., ImageNet, CIFAR-10) remains to be verified.
- **No hyperparameter optimization:** To maintain fairness, no model-specific hyperparameter tuning was performed, which may disadvantage some architectures.
- **Single hardware configuration:** Results may vary across different GPU models and hardware configurations.
- **Energy scope:** Only GPU energy during training is measured; inference energy, data loading, and evaluation costs are excluded.

6.3 Future Work

Several directions for future work emerge from this study:

- **Larger architectures and datasets:** Extending the comparison to larger parameter counts and more complex datasets such as CelebA-HQ, LSUN, and ImageNet.
- **Sampling energy:** Measuring the energy cost of inference/sampling in addition to training, which is particularly relevant for diffusion models that require many sequential denoising steps at inference time.
- **Newer model families:** Including score-based models, flow matching, consistency models, and latent diffusion models in the comparison.
- **Hardware-aware analysis:** Comparing energy efficiency across different GPU architectures (e.g., A100, H100, consumer GPUs) to understand how hardware choice affects the relative efficiency rankings.

Bibliography

[1] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[2] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. ICLR*, 2014.

[3] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[4] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. ICLR*, 2016.

[5] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp. 3730–3738, 2015.

[9] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[10] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. ACL*, pp. 3645–3650, 2019.

[11] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[12] CodeCarbon contributors, "CodeCarbon: Track and reduce CO_2 emissions from compute," https://codecarbon.io/, 2020.